

DSA in Java — 500 Practice Problems

20 topics × 25 problems, ordered easy → hard within each topic. Solve in plain Java (no library shortcuts for the core logic — implement the algorithm yourself, then compare against java.util where relevant). Aim for both a brute-force and an optimized solution on the "medium/hard" items in each section.

1. Arrays — Foundations

1. Find the second largest element in an array in a single pass.
2. Reverse an array in place without using extra space.
3. Rotate an array to the right by K steps, in place, in $O(n)$ time and $O(1)$ space.
4. Move all zeros in an array to the end while maintaining relative order of non-zero elements.
5. Find the missing number in an array of 1 to N using XOR.
6. Find the single non-repeating element in an array where every other element appears twice.
7. Find all pairs in an array that sum to a given target (two-sum, return all pairs).
8. Find the majority element (appears more than $n/2$ times) using Boyer-Moore voting.
9. Find the maximum product subarray.
10. Rearrange an array so that positive and negative numbers alternate.
11. Find the leaders in an array (elements greater than all elements to their right).
12. Find the equilibrium index of an array (left sum equals right sum).
13. Merge two sorted arrays in place without extra space.
14. Find the union and intersection of two unsorted arrays.
15. Find the minimum number of platforms needed for a railway station given arrival/departure times.
16. Implement Kadane's algorithm and extend it to return the actual subarray indices.
17. Find the Kth smallest and Kth largest element without full sorting.
18. Find all triplets in an array that sum to zero (3Sum), avoiding duplicates.
19. Find the trapping rainwater problem answer using the two-pointer approach.
20. Find the maximum sum of a subarray of size K (fixed window).
21. Find the next permutation of an array in place.
22. Find the longest consecutive sequence in an unsorted array in $O(n)$.
23. Sort an array of 0s, 1s, and 2s in one pass (Dutch National Flag problem).
24. Find the minimum number of swaps required to sort an array.
25. Find the maximum difference between two elements such that the larger appears after the smaller.

2. Arrays — Advanced & Matrix

1. Rotate an $N \times N$ matrix by 90 degrees in place.
2. Search for a target value in a row-wise and column-wise sorted matrix in $O(m+n)$.
3. Print a matrix in spiral order.
4. Find the maximum sum rectangle in a 2D matrix (Kadane's extended to 2D).
5. Set an entire row and column to zero if an element in the matrix is zero, in $O(1)$ extra space.
6. Find the row with the maximum number of 1s in a boolean matrix sorted row-wise.
7. Implement matrix multiplication and analyze its time complexity.
8. Find all elements that are peak elements in a 2D grid.
9. Count the number of islands in a 2D binary grid (DFS/BFS).
10. Find the shortest path in a binary matrix from top-left to bottom-right allowing 8-directional movement.
11. Implement flood fill algorithm for an image represented as a 2D array.
12. Find the number of distinct islands in a grid (shape matters, not just count).
13. Solve the "rotting oranges" problem using multi-source BFS.
14. Find the largest square of 1s in a binary matrix using DP.
15. Implement a word search in a 2D board (given a word, check if it exists via adjacent cells).
16. Find all paths from top-left to bottom-right in a grid with obstacles.
17. Compute the number of unique paths in a grid using DP and combinatorics, and compare approaches.
18. Find the median of two sorted arrays in $O(\log(\min(m,n)))$.
19. Merge overlapping intervals given as a 2D array.
20. Find the minimum number of intervals to remove to make the rest non-overlapping.
21. Implement a sparse matrix multiplication optimized for zero-skipping.
22. Find the largest rectangle of 1s in a binary matrix (histogram-based approach).
23. Find the number of ways to tile a $2 \times N$ board using 1×2 tiles (DP).
24. Implement Conway's Game of Life on an in-place 2D grid.
25. Find the shortest bridge connecting two islands in a grid (BFS + multi-source BFS).

3. Strings

1. Check if a string is a palindrome ignoring case and non-alphanumeric characters.
2. Reverse the words in a sentence in place (without using split-and-join if possible).
3. Find the first non-repeating character in a string.
4. Check if two strings are anagrams of each other.
5. Group a list of strings into anagram groups.
6. Implement string compression (e.g., "aaabbc" -> "a3b2c1").

7. Find the longest substring without repeating characters using the sliding window technique.
8. Find the longest palindromic substring using expand-around-center and compare with DP approach.
9. Implement the KMP (Knuth-Morris-Pratt) string matching algorithm from scratch.
10. Implement the Rabin-Karp string matching algorithm using rolling hash.
11. Check if one string is a rotation of another using a single string-contains check.
12. Find all permutations of a string (with and without duplicates).
13. Implement a basic run-length encoding and decoding function.
14. Find the minimum number of insertions to make a string a palindrome.
15. Find the longest common prefix among an array of strings.
16. Implement wildcard pattern matching (supporting '?' and '*').
17. Implement regular expression matching (supporting '.' and '*') from scratch.
18. Find the edit distance (Levenshtein distance) between two strings using DP.
19. Check if a string can be segmented into space-separated dictionary words (word break problem).
20. Find all valid parenthesis combinations for N pairs of brackets.
21. Implement a basic text justification algorithm (given words and a line width).
22. Find the minimum window substring in a string that contains all characters of another string.
23. Count and print all substrings of a string that are palindromes.
24. Implement string-to-integer (atoi) conversion handling edge cases manually.
25. Find the longest repeating substring in a string using suffix-based techniques.

4. Two Pointers & Sliding Window

1. Find a pair with a given sum in a sorted array using two pointers.
2. Find the maximum sum of a subarray of size K using a sliding window.
3. Find the smallest subarray with a sum greater than or equal to a given value.
4. Find the longest substring with at most K distinct characters.
5. Find all anagrams of a pattern string within a larger string using a sliding window.
6. Implement the "container with most water" problem using two pointers.
7. Remove duplicates from a sorted array in place using two pointers.
8. Find the length of the longest subarray with equal number of 0s and 1s.
9. Find the maximum number of fruits that can be collected in two baskets (longest subarray with at most 2 distinct types).
10. Find the smallest window in a string containing all characters of another string (minimum window substring).
11. Find all subarrays with a given XOR value.
12. Find the count of subarrays with exactly K distinct elements (using at-most-K trick).

13. Find the longest subarray with sum at most K (positive and negative numbers).
14. Find the maximum average subarray of size K.
15. Merge two sorted linked lists using a two-pointer technique.
16. Find the middle of a linked list using slow/fast pointers.
17. Detect a cycle in a linked list and find its starting node (Floyd's algorithm).
18. Partition an array around a pivot using the Dutch National Flag approach (three-way partition).
19. Find triplets that sum to a target using sorting plus two pointers.
20. Find quadruplets that sum to a target (4Sum) efficiently.
21. Trap rainwater using the two-pointer approach and compare with the prefix-max approach.
22. Find the longest mountain in an array using expand-from-peak with two pointers.
23. Find the maximum length of a subarray with a sum of zero.
24. Implement a sliding window maximum using a deque.
25. Find the shortest subarray to remove to make the rest of the array sorted.

5. Linked List

1. Implement a singly linked list from scratch with insert, delete, and search operations.
2. Reverse a singly linked list iteratively and recursively.
3. Detect and remove a cycle in a linked list.
4. Find the intersection point of two linked lists.
5. Find the Nth node from the end of a linked list in a single pass.
6. Merge two sorted linked lists into one sorted linked list.
7. Merge K sorted linked lists using a min-heap.
8. Reverse a linked list in groups of K.
9. Check if a linked list is a palindrome without using extra space.
10. Remove duplicates from an unsorted linked list.
11. Implement a doubly linked list with insert, delete, and reverse operations.
12. Flatten a multilevel doubly linked list.
13. Add two numbers represented as linked lists (digits in reverse order).
14. Clone a linked list with random pointers.
15. Rotate a linked list to the right by K places.
16. Partition a linked list around a value X, keeping relative order.
17. Find the starting node of a cycle in a linked list and the cycle's length.
18. Sort a linked list using merge sort (no extra array).
19. Implement an LRU cache using a doubly linked list and a hash map.
20. Implement an LFU cache using linked lists/hash maps.
21. Swap every two adjacent nodes in a linked list without swapping values.
22. Remove the Nth node from the end of a linked list in one pass.

23. Convert a binary tree to a doubly linked list in-order.
24. Implement a skip list supporting insert, search, and delete.
25. Find the fractional/decimal representation of a linked-list-based fraction with cycle detection.

6. Stack & Queue

1. Implement a stack using arrays and using a linked list.
2. Implement a queue using two stacks.
3. Implement a stack using two queues.
4. Check for balanced parentheses in an expression using a stack.
5. Evaluate a postfix expression using a stack.
6. Convert an infix expression to postfix using a stack.
7. Implement a min-stack that supports push, pop, top, and getMin in $O(1)$.
8. Find the next greater element for every element in an array using a stack.
9. Find the next smaller element for every element in an array using a stack.
10. Implement a stock span problem solution using a stack.
11. Find the largest rectangle in a histogram using a stack.
12. Implement a circular queue using an array.
13. Implement a deque (double-ended queue) from scratch.
14. Design a max/min sliding window using a deque.
15. Implement an LRU cache using only a queue-like structure (compare vs linked-list+map approach).
16. Simplify a Unix-style file path using a stack.
17. Implement a basic calculator that evaluates expressions with +, -, parentheses using a stack.
18. Implement a basic calculator supporting *, /, +, - with operator precedence.
19. Sort a stack using only another stack (no extra array).
20. Implement a queue that supports getMax/getMin in $O(1)$ amortized.
21. Design a browser history feature using two stacks (back/forward navigation).
22. Implement an undo/redo mechanism using two stacks.
23. Check if a sequence of pushes and pops on a stack is valid, given push order and a candidate pop order.
24. Implement a priority queue from scratch using a binary heap array (no `java.util.PriorityQueue`).
25. Design a task scheduler that processes tasks with cooldown periods using a queue/heap combination.

7. Binary Search

1. Implement binary search iteratively and recursively.
2. Find the first and last occurrence of a target in a sorted array.

3. Find the number of occurrences of a target in a sorted array.
4. Search in a rotated sorted array (no duplicates).
5. Search in a rotated sorted array with duplicates.
6. Find the minimum element in a rotated sorted array.
7. Find the peak element in an array (element greater than its neighbors) using binary search.
8. Find the square root of a number without using built-in functions, using binary search.
9. Find the Kth smallest element in a sorted matrix using binary search on value.
10. Find the capacity to ship packages within D days using binary search on the answer.
11. Find the minimum number of days to make M bouquets from a garden using binary search on the answer.
12. Find the Nth root of a number using binary search.
13. Find the allocation of minimum number of pages among students using binary search on the answer.
14. Implement binary search on an answer for the "split array largest sum" problem.
15. Find the median of two sorted arrays using binary search partitioning.
16. Find an element in an infinite sorted array (unknown size) using exponential + binary search.
17. Find the smallest divisor such that the sum of divisions is less than or equal to a threshold.
18. Implement ternary search for finding the maximum of a unimodal function.
19. Find the closest element to a target in a sorted array.
20. Find the number of times a sorted array has been rotated.
21. Find the position to insert a target in a sorted array (lower bound / upper bound implementation).
22. Find the aggressive cows placement problem (maximize minimum distance) using binary search.
23. Find the koko eating bananas minimum speed using binary search on the answer.
24. Find the weighted median of two sorted arrays with binary search.
25. Implement binary search over a 2D matrix treated as a flattened sorted array.

8. Recursion & Backtracking

1. Implement factorial, Fibonacci, and power functions recursively with memoization.
2. Print all subsets (power set) of a given set using recursion.
3. Print all permutations of an array/string using backtracking.
4. Solve the N-Queens problem and print all valid board configurations.
5. Solve the Sudoku solver problem using backtracking.
6. Solve the Rat in a Maze problem (find all paths from source to destination).
7. Generate all valid combinations of balanced parentheses for N pairs.

8. Solve the combination sum problem (find all combinations that sum to a target, with repetition allowed).
9. Solve the combination sum II problem (no repetition, avoid duplicate combinations).
10. Solve the subset sum problem using backtracking and compare with the DP approach.
11. Implement the M-coloring problem for graph coloring using backtracking.
12. Solve the word search problem in a 2D board using backtracking with pruning.
13. Generate all possible palindrome partitions of a string.
14. Solve the "restore IP addresses" problem given a string of digits.
15. Implement the knight's tour problem using backtracking.
16. Solve the Hamiltonian path/cycle problem in a graph using backtracking.
17. Generate all possible unique subsets when the input array contains duplicates.
18. Solve the "expression add operators" problem (insert +, -, * to reach a target).
19. Implement backtracking to solve a crossword puzzle fill problem.
20. Solve the "beautiful arrangement" problem using backtracking with pruning.
21. Implement a recursive solution to flatten a nested list structure.
22. Solve the tower of Hanoi problem and print the sequence of moves.
23. Implement recursive backtracking for generating all k-length combinations from n elements.
24. Solve the "partition to K equal sum subsets" problem using backtracking.
25. Implement a recursive solution for matchstick/strings to squares partitioning problems.

9. Trees (Binary Tree)

1. Implement a binary tree with insert, in-order, pre-order, and post-order traversals (recursive).
2. Implement level-order traversal (BFS) of a binary tree using a queue.
3. Implement in-order, pre-order, and post-order traversal iteratively using a stack.
4. Find the height/depth of a binary tree.
5. Find the diameter of a binary tree (longest path between any two nodes).
6. Check if a binary tree is height-balanced.
7. Check if two binary trees are identical or mirror images of each other.
8. Find the lowest common ancestor (LCA) of two nodes in a binary tree.
9. Find the maximum path sum in a binary tree (path can start/end anywhere).
10. Serialize and deserialize a binary tree.
11. Convert a binary tree to its mirror image.
12. Find all root-to-leaf paths in a binary tree.
13. Check if a binary tree is a sum tree (each node's value equals the sum of its subtree).
14. Find the vertical order traversal of a binary tree.
15. Find the boundary traversal of a binary tree (left boundary, leaves, right boundary).

16. Find the zigzag (spiral) level order traversal of a binary tree.
17. Construct a binary tree from in-order and pre-order traversal arrays.
18. Construct a binary tree from in-order and post-order traversal arrays.
19. Find the width of a binary tree at its widest level.
20. Flatten a binary tree to a linked list in place (pre-order).
21. Find the kth ancestor of a node in a binary tree.
22. Count the number of nodes in a complete binary tree in less than $O(n)$ time.
23. Find duplicate subtrees in a binary tree.
24. Implement a Morris traversal for in-order traversal without recursion or a stack.
25. Find all nodes at distance K from a given node in a binary tree.

10. Binary Search Trees

1. Implement a BST with insert, delete, and search operations.
2. Validate whether a given binary tree is a valid BST.
3. Find the kth smallest and kth largest element in a BST.
4. Find the lowest common ancestor of two nodes in a BST (using BST properties for efficiency).
5. Convert a sorted array into a height-balanced BST.
6. Convert a BST into a sorted doubly linked list.
7. Find the floor and ceiling of a given value in a BST.
8. Delete a node from a BST while maintaining BST properties.
9. Find the inorder successor and predecessor of a node in a BST.
10. Merge two BSTs into a single balanced BST.
11. Find if there exists a pair in a BST with a given sum (two-sum on a BST).
12. Construct a BST from a preorder traversal array.
13. Find the closest value to a target in a BST.
14. Count the number of BSTs that can be formed with N nodes (Catalan number relation).
15. Recover a BST where exactly two nodes have been swapped.
16. Find the largest BST subtree within a binary tree.
17. Implement a self-balancing BST (AVL tree) with rotations on insert.
18. Implement a Red-Black tree insert operation with recoloring and rotations.
19. Find the median of a stream of numbers using two BSTs/heaps and compare approaches.
20. Implement range queries (count of elements between L and R) on a BST.
21. Find all nodes within a given range [L, R] in a BST.
22. Serialize and deserialize a BST more efficiently than a general binary tree.
23. Convert a BST to a balanced BST (rebalance an unbalanced BST).
24. Implement a Trie-like BST for storing and searching strings lexicographically.

25. Find the two nodes in a BST whose values sum closest to a target.

11. Heaps & Priority Queue

1. Implement a binary min-heap and max-heap from scratch (array-based) with insert and extract-min/max.
2. Implement heap sort from scratch.
3. Find the Kth largest element in an unsorted array using a heap.
4. Find the K closest points to the origin using a max-heap.
5. Merge K sorted arrays/lists using a min-heap.
6. Find the median of a running data stream using two heaps (max-heap + min-heap).
7. Implement a top-K frequent elements solution using a heap.
8. Design a task scheduler with cooldown using a max-heap.
9. Find the smallest range covering elements from K sorted lists using a heap.
10. Implement a priority queue-based Dijkstra's algorithm for shortest paths.
11. Find the Kth smallest element in a sorted matrix using a heap.
12. Reorganize a string so that no two adjacent characters are the same, using a max-heap.
13. Implement a heap-based solution for the "ugly number" sequence generation.
14. Find the maximum sum of K non-overlapping subarrays using a heap-assisted approach.
15. Implement a double-ended priority queue (min-max heap).
16. Connect N ropes with minimum cost using a min-heap.
17. Find the sliding window median using two heaps with lazy deletion.
18. Implement a heap-based solution for the "IPO" problem (maximize capital with K projects).
19. Design a leaderboard system using a heap for top-K score retrieval.
20. Find the frequency-sorted string reconstruction using a max-heap.
21. Implement a heap-based Huffman encoding algorithm.
22. Find the minimum cost to connect all points in a plane (Prim's/Kruskal's using a heap).
23. Implement an event scheduler that finds the maximum number of non-overlapping events using a heap.
24. Find the Kth largest element in a stream of numbers (maintain a running heap).
25. Implement a heap-based solution for meeting room scheduling (minimum meeting rooms required).

12. Graphs — BFS/DFS Foundations

1. Implement a graph using an adjacency list and adjacency matrix representation.
2. Implement BFS traversal of a graph and detect connected components.
3. Implement DFS traversal of a graph (recursive and iterative with an explicit stack).
4. Detect a cycle in an undirected graph using BFS/DFS.

5. Detect a cycle in a directed graph using DFS (colored/visited-state approach).
6. Implement topological sort using DFS.
7. Implement topological sort using Kahn's algorithm (BFS-based).
8. Check if a graph is bipartite using BFS/DFS coloring.
9. Find the number of connected components in an undirected graph.
10. Find the number of islands in a grid using DFS/BFS (graph framing of a matrix problem).
11. Implement a clone-graph function (deep copy of a graph with cycles).
12. Find all paths between two nodes in a directed graph.
13. Find the shortest path in an unweighted graph using BFS.
14. Implement a flood-fill style multi-source BFS for the "rotting oranges" style problem.
15. Find bridges in a graph (edges whose removal disconnects the graph) using Tarjan's algorithm.
16. Find articulation points (cut vertices) in a graph using DFS.
17. Find strongly connected components using Kosaraju's algorithm.
18. Find strongly connected components using Tarjan's algorithm.
19. Implement a word ladder solution using BFS (transform one word to another one letter at a time).
20. Find if a path exists between two nodes given a list of edges (union-find or BFS/DFS).
21. Implement a graph-based course scheduling validity check (detect cycle in prerequisites graph).
22. Find the order of courses to take given prerequisites (topological sort application).
23. Implement a snake and ladder game solver using BFS (minimum number of dice throws).
24. Find the shortest path to get all keys in a grid maze using BFS with state tracking.
25. Implement a graph coloring solution to check if a graph can be colored with M colors.

13. Graphs — Advanced Algorithms

1. Implement Dijkstra's algorithm using a priority queue for single-source shortest paths.
2. Implement the Bellman-Ford algorithm and detect negative weight cycles.
3. Implement the Floyd-Warshall algorithm for all-pairs shortest paths.
4. Implement Prim's algorithm for finding the minimum spanning tree.
5. Implement Kruskal's algorithm for finding the minimum spanning tree using union-find.
6. Implement a union-find (disjoint set) data structure with path compression and union by rank.
7. Find the number of provinces (connected components) using union-find.
8. Detect a cycle in a graph using union-find.
9. Implement Kruskal's algorithm to find the minimum cost to connect all cities.

10. Find the critical connections (bridges) in a network using Tarjan's algorithm.
11. Implement the A* search algorithm for shortest path with a heuristic.
12. Find the cheapest flights within K stops using a modified Bellman-Ford/BFS.
13. Implement a network delay time solution using Dijkstra's algorithm.
14. Find the minimum spanning tree cost and compare Prim's vs Kruskal's performance on dense vs sparse graphs.
15. Implement a solution for "swim in rising water" using Dijkstra's/binary search + BFS.
16. Find the shortest path in a weighted graph with at most K edges.
17. Implement a solution to reconstruct an itinerary from a list of flight tickets (Eulerian path).
18. Find the minimum number of semesters to complete all courses with prerequisites (BFS level-based topological sort).
19. Implement a solution for the "alien dictionary" problem (topological sort on character ordering).
20. Find the maximum flow in a flow network using the Ford-Fulkerson algorithm.
21. Implement the Edmonds-Karp algorithm (BFS-based Ford-Fulkerson) for maximum flow.
22. Find the minimum cut in a flow network using max-flow min-cut theorem.
23. Implement bipartite matching using the Hopcroft-Karp or augmenting path algorithm.
24. Find the shortest path visiting all nodes in a graph (Hamiltonian-path style with bitmask DP).
25. Implement a graph-based solution for finding redundant connections (cycle-causing edge) using union-find.

14. Dynamic Programming — 1D & Classic

1. Solve the Fibonacci sequence using memoization and tabulation, and compare with the naive recursive approach.
2. Solve the climbing stairs problem (count distinct ways to reach the top).
3. Solve the house robber problem (maximum sum of non-adjacent elements).
4. Solve the house robber II problem (circular array variant).
5. Solve the coin change problem (minimum number of coins to make a target amount).
6. Solve the coin change II problem (count the number of ways to make a target amount).
7. Solve the longest increasing subsequence problem using $O(n^2)$ DP and $O(n \log n)$ patience sorting.
8. Solve the maximum product subarray problem using DP.
9. Solve the word break problem using DP (string segmentation into dictionary words).
10. Solve the decode ways problem (count ways to decode a digit string into letters).

11. Solve the perfect squares problem (minimum number of perfect squares summing to N).
12. Solve the jump game problem (can you reach the last index) and the minimum jumps variant.
13. Solve the partition equal subset sum problem using DP.
14. Solve the target sum problem (count ways to assign +/- to reach a target).
15. Solve the longest palindromic subsequence problem using DP.
16. Solve the minimum number of deletions to make a string a palindrome.
17. Solve the maximum sum of non-adjacent elements in a circular array.
18. Solve the paint house problem (minimum cost to paint houses with no two adjacent the same color).
19. Solve the delete and earn problem (transform to house robber).
20. Solve the ones and zeroes problem (0/1 knapsack with two constraints).
21. Solve the counting bits problem using DP over bit patterns.
22. Solve the integer break problem (maximize product of integers summing to N).
23. Solve the wildcard/regex matching problem using 2D DP.
24. Solve the minimum cost to cut a stick problem using interval DP.
25. Solve the burst balloons problem using interval DP.

15. Dynamic Programming — 2D & Advanced

1. Solve the 0/1 knapsack problem using 2D DP and then optimize to 1D.
2. Solve the unbounded knapsack problem (rod cutting problem variant).
3. Solve the longest common subsequence problem and reconstruct the actual subsequence.
4. Solve the longest common substring problem using DP.
5. Solve the edit distance problem (Levenshtein distance) using 2D DP.
6. Solve the distinct subsequences problem (count ways one string appears as a subsequence of another).
7. Solve the interleaving strings problem using 2D DP.
8. Solve the minimum path sum problem in a grid using DP.
9. Solve the unique paths problem with obstacles using DP.
10. Solve the maximal square problem (largest square of 1s in a binary matrix).
11. Solve the maximal rectangle problem (largest rectangle of 1s in a binary matrix).
12. Solve the cherry pickup problem (two paths through a grid maximizing collected cherries).
13. Solve the matrix chain multiplication problem using interval DP.
14. Solve the egg drop puzzle using DP.
15. Solve the palindrome partitioning II problem (minimum cuts for palindrome partitioning).

16. Solve the scramble string problem using DP with memoization.
17. Solve the shortest common supersequence problem using DP.
18. Solve the DP-based Boolean parenthesization problem (count ways to parenthesize to get a target result).
19. Solve the DP-based longest arithmetic subsequence problem.
20. Solve the maximum profit in job scheduling problem using DP + binary search.
21. Solve the DP-based tiling problem (count ways to tile a board with dominoes/trominoes).
22. Solve the DP-based stock buy/sell problem with at most K transactions.
23. Solve the DP-based stock buy/sell problem with a cooldown period.
24. Solve the regular expression matching problem using 2D DP (supporting '.' and '*').
25. Solve the bitmask DP-based traveling salesman problem for small N.

16. Greedy Algorithms

1. Solve the activity selection problem (maximum number of non-overlapping activities).
2. Solve the fractional knapsack problem using a greedy approach.
3. Solve the job sequencing problem with deadlines to maximize profit.
4. Solve the minimum number of coins problem using a greedy approach and identify when it fails vs DP.
5. Implement Huffman coding using a greedy + heap approach.
6. Solve the gas station problem (can you complete a circular tour).
7. Solve the candy distribution problem (minimum candies with rating constraints).
8. Solve the jump game II problem (minimum jumps to reach the end) using a greedy approach.
9. Solve the minimum number of platforms problem using a greedy interval approach.
10. Solve the non-overlapping intervals problem (minimum removals to make intervals non-overlapping).
11. Solve the merge intervals problem using a greedy sort-then-merge approach.
12. Solve the insert interval problem (insert a new interval into a sorted list of intervals).
13. Solve the partition labels problem (partition a string into as many parts as possible).
14. Solve the task scheduler problem with cooldown using a greedy + heap approach.
15. Solve the minimum number of arrows to burst balloons problem.
16. Solve the assign cookies problem (maximize satisfied children with greedy matching).
17. Solve the boats to save people problem using a two-pointer greedy approach.
18. Solve the reorganize string problem using a greedy + heap approach.
19. Solve the minimum deletions to make character frequencies unique using a greedy approach.
20. Solve the two-city scheduling problem to minimize total cost.

21. Solve the queue reconstruction by height problem using a greedy + insertion approach.
22. Solve the greedy Dijkstra-style approach for the "cheapest flights" simplification (single stop case).
23. Solve the maximum number of events that can be attended using a greedy + heap approach.
24. Solve the minimum cost to hire K workers problem using a greedy + heap approach.
25. Solve the greedy interval point cover problem (minimum number of points to cover all intervals).

17. Bit Manipulation

1. Check if a number is a power of two using bitwise operations.
2. Count the number of set bits (1s) in an integer (Brian Kernighan's algorithm).
3. Find the single number in an array where every other number appears twice, using XOR.
4. Find the two single numbers in an array where every other number appears twice.
5. Find the single number in an array where every other number appears three times.
6. Swap two numbers without using a temporary variable, using XOR.
7. Find the missing number in an array of 0 to N using XOR/bit tricks.
8. Implement addition of two integers without using the + operator, using bitwise operations.
9. Count the total number of set bits from 1 to N.
10. Find the position of the rightmost set bit in a number.
11. Check if a number has alternating bits.
12. Implement a subset generation using bitmasking instead of recursion.
13. Find the maximum XOR of two numbers in an array using a bitwise trie.
14. Find the maximum XOR subarray/subset using a trie-based approach.
15. Implement a bitmask DP solution for the "smallest sufficient team" problem.
16. Reverse the bits of a 32-bit unsigned integer.
17. Find the Hamming distance between two integers.
18. Implement a bitset-based sieve of Eratosthenes for finding primes efficiently.
19. Find all subsets of a set using bitmasking and print them in order.
20. Implement a bitwise trick to check if two integers have opposite signs.
21. Find the number of bit flips required to convert one integer to another.
22. Implement gray code generation using bit manipulation.
23. Find the maximum AND value of a pair in an array less than a given limit.
24. Implement a bitmask-based solution to the "partition to K equal sum subsets" as an alternative to backtracking.
25. Find the minimum number of operations to reduce a number to 1 using bitwise/greedy operations (halve if even, subtract 1 if odd).

18. Trie & String Algorithms

1. Implement a Trie (prefix tree) with insert, search, and startsWith operations.
2. Implement a word dictionary supporting add and search with wildcard '.' matching.
3. Implement an autocomplete system using a Trie with frequency-based ranking.
4. Find the longest word in a dictionary that can be built one character at a time using a Trie.
5. Implement a Trie-based solution for replacing words with their shortest root in a sentence.
6. Find the maximum XOR of two numbers in an array using a binary Trie.
7. Implement a Trie-based solution for the word search II problem (find multiple words in a board).
8. Implement a suffix array construction for a string and use it for pattern matching.
9. Implement the Z-algorithm for pattern matching.
10. Implement the Manacher's algorithm for finding the longest palindromic substring in $O(n)$.
11. Implement a suffix tree (conceptually) or use a suffix array for finding the longest repeated substring.
12. Implement a Trie-based solution for counting distinct substrings of a string.
13. Implement string hashing (polynomial rolling hash) and use it for substring comparison.
14. Solve the "longest duplicate substring" problem using binary search + rolling hash.
15. Implement a Trie to solve the "concatenated words" problem (words formed by other words in the dictionary).
16. Implement a Trie-based IP routing table lookup (longest prefix match).
17. Implement the Aho-Corasick algorithm for multi-pattern string matching.
18. Solve the "palindrome pairs" problem using a Trie.
19. Implement a Trie-based spell checker with edit-distance-based suggestions.
20. Solve the "stream of characters" problem (check if a suffix matches any word) using a Trie.
21. Implement a compressed Trie (radix tree) for space-efficient prefix storage.
22. Solve the "design search autocomplete system" problem combining a Trie with a heap for top suggestions.
23. Implement a Trie-based solution for finding all words that match a pattern with wildcards.
24. Solve the "maximum XOR with an element from array" problem using an offline Trie approach.
25. Implement a Trie-based solution to find the shortest unique prefix for every word in an array.

19. Advanced Data Structures

1. Implement a Fenwick Tree (Binary Indexed Tree) supporting point update and prefix sum query.
2. Implement a Segment Tree supporting range sum queries and point updates.
3. Extend the Segment Tree to support range updates using lazy propagation.
4. Implement a Segment Tree for range minimum/maximum queries.
5. Implement a Fenwick Tree for range update and point query (using difference arrays).
6. Implement a 2D Fenwick Tree for 2D range sum queries.
7. Implement a Disjoint Set Union (Union-Find) with path compression and union by rank/size.
8. Implement a Sparse Table for $O(1)$ range minimum queries with $O(n \log n)$ preprocessing.
9. Design and implement an LRU cache from scratch (HashMap + doubly linked list).
10. Design and implement an LFU cache from scratch.
11. Design a data structure supporting insert, delete, and getRandom in $O(1)$ average time.
12. Design a data structure for a leaderboard supporting score updates and top-K queries.
13. Implement a skip list supporting search, insert, and delete in $O(\log n)$ expected time.
14. Design a time-based key-value store supporting set and get with timestamps.
15. Implement a persistent (versioned) array/stack data structure conceptually.
16. Design a data structure that supports insert, remove, and getRandom with duplicates allowed.
17. Implement an order-statistics tree supporting rank and select operations.
18. Design a data structure for range frequency queries in an array (count occurrences of a value in a range).
19. Implement a Segment Tree supporting range GCD queries.
20. Design a data structure for the "median of a stream" problem using two heaps and compare with a balanced BST.
21. Implement a Trie-based Fenwick Tree hybrid for counting distinct elements in ranges (offline queries).
22. Design a rate limiter data structure (sliding window counter) with $O(1)$ amortized operations.
23. Implement a Merkle-tree-like structure conceptually for verifying data integrity in chunks.
24. Design a data structure supporting insert, delete, and find-median in $O(\log n)$.
25. Implement a Fenwick Tree-based solution for counting inversions in an array.

20. Math, Number Theory & Mixed Hard Problems

1. Implement the Sieve of Eratosthenes to find all primes up to N .
2. Implement a segmented sieve for finding primes in a large range efficiently.

3. Implement fast exponentiation (modular exponentiation) in $O(\log n)$.
 4. Implement the Euclidean algorithm for GCD and extend it to the extended Euclidean algorithm.
 5. Find all divisors of a number efficiently in $O(\sqrt{n})$.
 6. Check if a number is prime using trial division optimized to $O(\sqrt{n})$.
 7. Implement modular inverse computation using Fermat's little theorem.
 8. Solve the "count primes less than N" problem and analyze the time complexity.
 9. Implement prime factorization of a number in $O(\sqrt{n})$ and using a precomputed smallest-prime-factor sieve.
 10. Solve the "power of three/four" problem using mathematical properties (no loops/recursion allowed variant).
 11. Implement the Chinese Remainder Theorem for solving simultaneous congruences.
 12. Solve the "excel sheet column number/title" conversion problems (base-26 style conversion).
 13. Implement matrix exponentiation to compute the Nth Fibonacci number in $O(\log n)$.
 14. Solve the "count of trailing zeroes in factorial" problem using Legendre's formula.
 15. Implement the "pow(x, n)" function handling negative exponents efficiently.
 16. Solve the "reverse integer" and "palindrome number" problems handling overflow correctly.
 17. Solve the "happy number" problem using cycle detection (Floyd's algorithm on a sequence).
 18. Solve the "roman numeral" to integer and integer to roman numeral conversion problems.
 19. Implement the "random pick with weight" problem using prefix sums and binary search.
 20. Solve the "count of range sum" problem using a merge-sort-based approach.
 21. Implement reservoir sampling to randomly select K items from a stream of unknown size.
 22. Solve the "shuffle an array" problem implementing the Fisher-Yates algorithm.
 23. Implement a solution for the "find the duplicate number" problem using Floyd's cycle detection on array-as-linked-list.
 24. Solve the "median of a data stream" problem end-to-end as a capstone combining heaps, and compare with a Fenwick-tree-based order-statistics approach.
 25. Design and implement a rate-limited, thread-safe LRU + TTL cache combining several data structures from this list as a final integration exercise.
-

How to use this

- **Pacing:** 25 problems/topic, 20 topics. At 10/day this is a 50-day sprint; at 15/day, ~33 days.

- **Sequencing:** Sections 1–8 build core technique (arrays, strings, two pointers, recursion) — don't skip these even if they feel basic, since later sections assume fluency here. Sections 9–13 (trees, BSTs, heaps, graphs) are where most mid-to-senior interviews concentrate. Sections 14–16 (DP, greedy) are typically the hardest bar to clear — expect to revisit problems. Sections 17–20 (bit manipulation, tries, advanced structures, math) round out edge cases and are common in staff/principal-level or competitive-programming-style rounds.
- **Practice discipline:** For each problem, first write the brute-force solution and state its complexity, then optimize. Track problems you solved sub-optimally and revisit them after finishing a section, rather than only after finishing the whole set.